

ASTRAZIONE DEL CONTROLLO

↳ identificare cosa è necessario descrivere -
mantenere pubblica e cosa possiamo
nascondere



così possiamo focalizzare la programmazione
sulle questioni rilevanti

ottimizzare
implementazione

ridurre
errori

Migliorare
leggibilità

⇒ Introduce legami che vanno gestiti

⇒ Astrazione del controllo ⇒ comandi

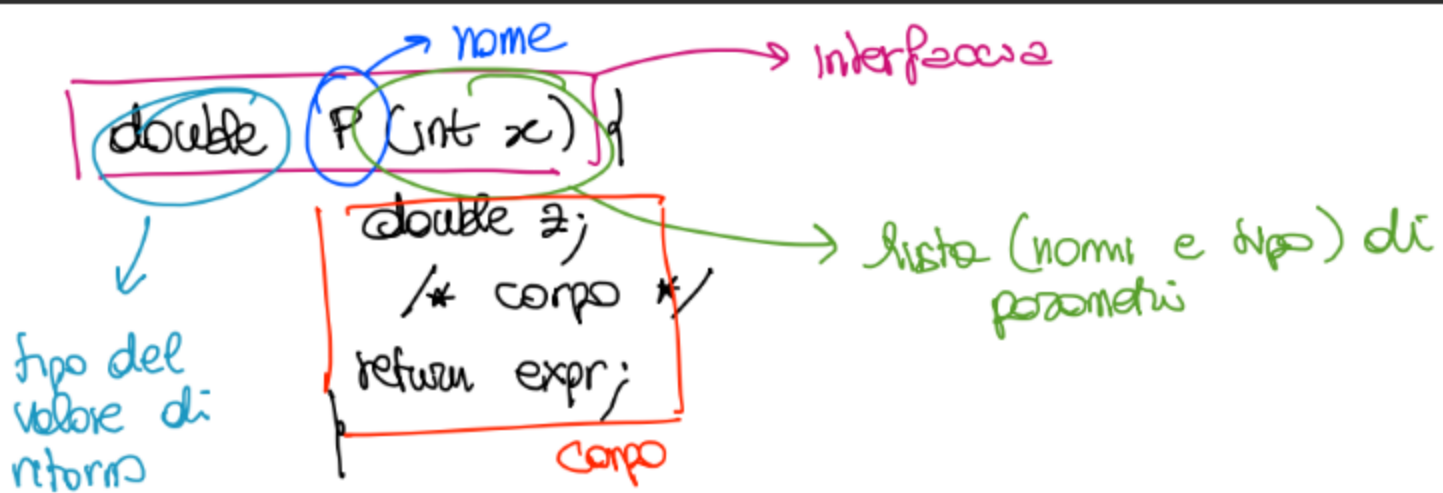
Nome ⇒ Riferisce ad un insieme di comandi

Nome → [Codice che esegue sempre
nello stesso modo]

Parametri ⇒ permettono di usare un pezzo
di codice su valori (input) differenti
⇒ eseguiamo computazioni simili
richiamate usando lo stesso nome

+ ^{tipo} Risultato ≡ interfaccia

La parte nascosta dell'astrazione → corpo della
procedura
≡ codice eseguito



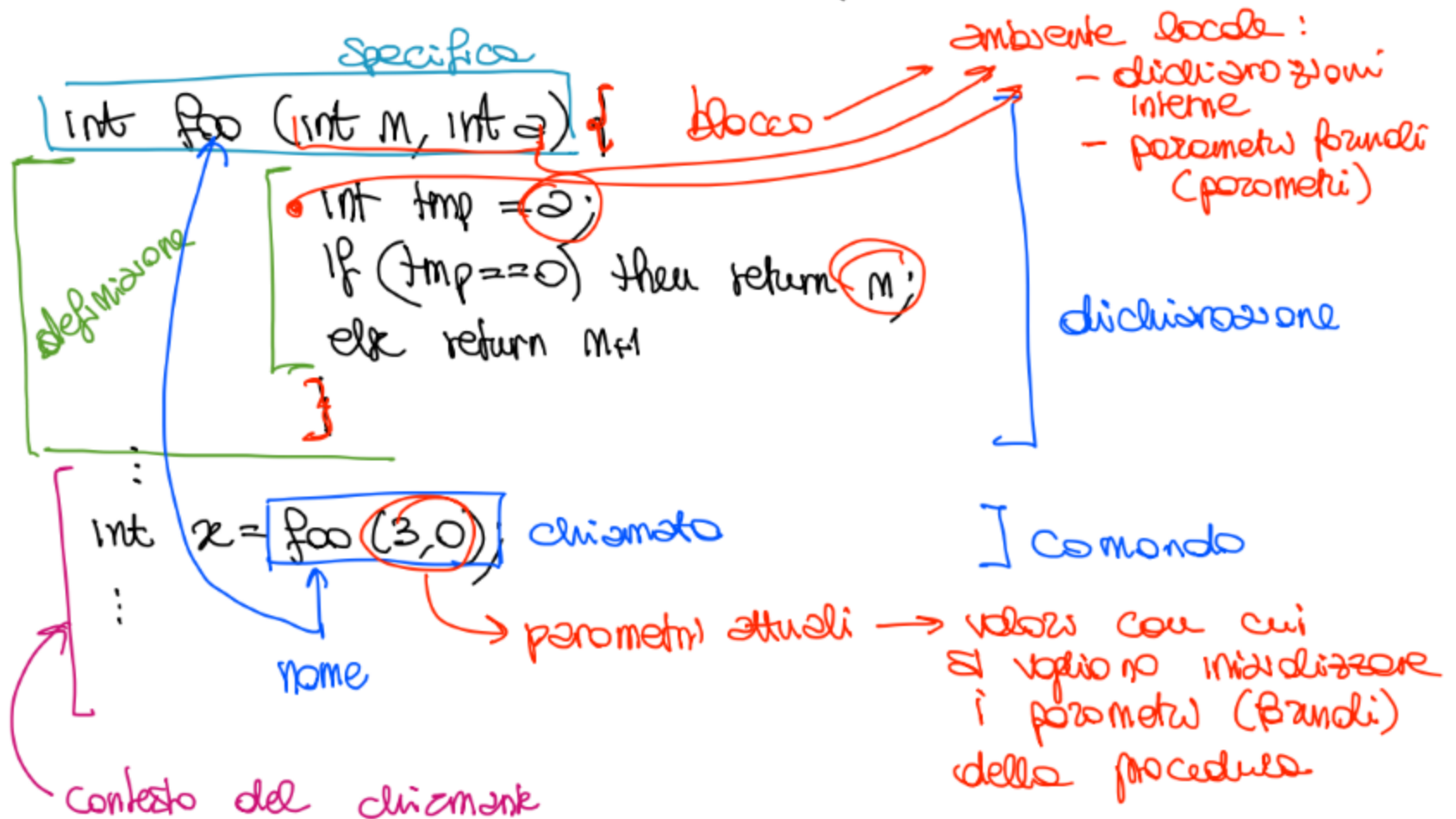
Per definire procedure il PL deve avere strumenti sintattici

→ per specificare (definire) P (interfaccia)
 → scrivere P

definitore
completa di P

↓
Dichiarazioni

→ utilizzare P (chiamare l'esecuzione del corpo di P) → Comandi



```
int foo (int m) {
```

```
    int tmp = 2
```

```
    :
```

```
}
```

→ identificatore non locale

→ identificatore libero
che trovare significato (legame)
nel contesto

PROBLEMA: Nel contesto (chiamante)
potrebbero esserci più
definizioni di x , quale
usiamo?

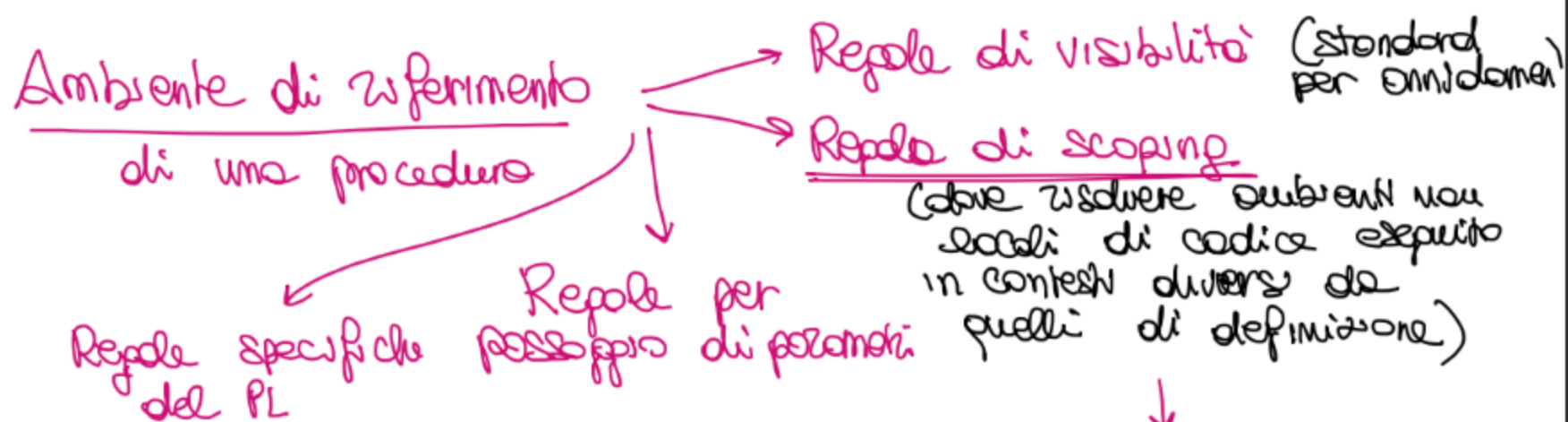
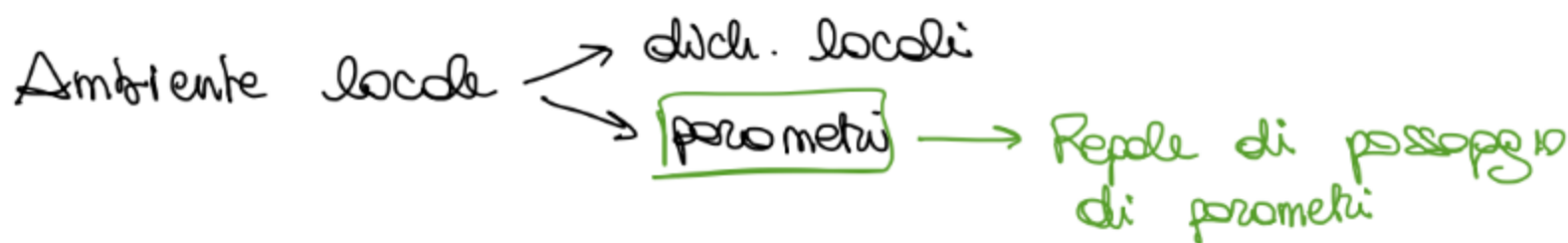
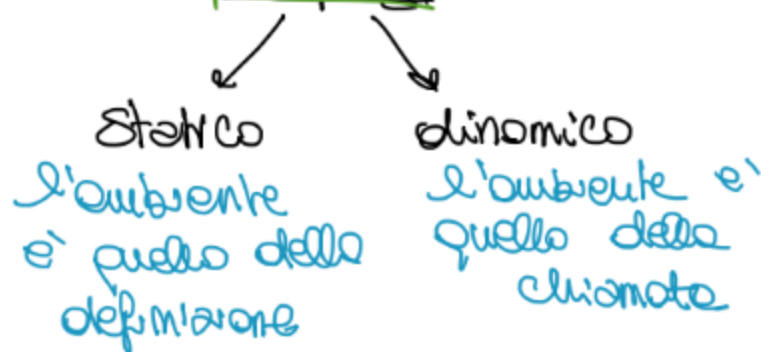
Una dichiarazione locale ad un blocco è visibile
nel blocco e in tutti i blocchi in questo annidati
e meno di 2 definizioni dello stesso identificatore



Regole di visibilità
non è sufficiente a
risolvere i riferimenti
quando usiamo le procedure
(e le procedure HA ambiente
non locale)

PROBLEMA: come può ambiente va usato come
riferimento per l'ambiente non locale
della procedura

→ Va definita la regola di scoping



↓
Regole di binding

int f (g funzione ...)
funzione come parametro di un'altra funzione

- def g ← scoping statico
 - passaggio di g ad f
 - esecuzione di g dentro f
- scoping dinamico


```

{ Int x = 10;
  void foo () {
    x++;
  }
}

```

definito

Ambiente non locale (x non è parametro formale e non è dichiarato nel blocco)

```

void fie () {
  Int x = 0;
  foo ();
}

fie ();

```

eseguito

x = 1

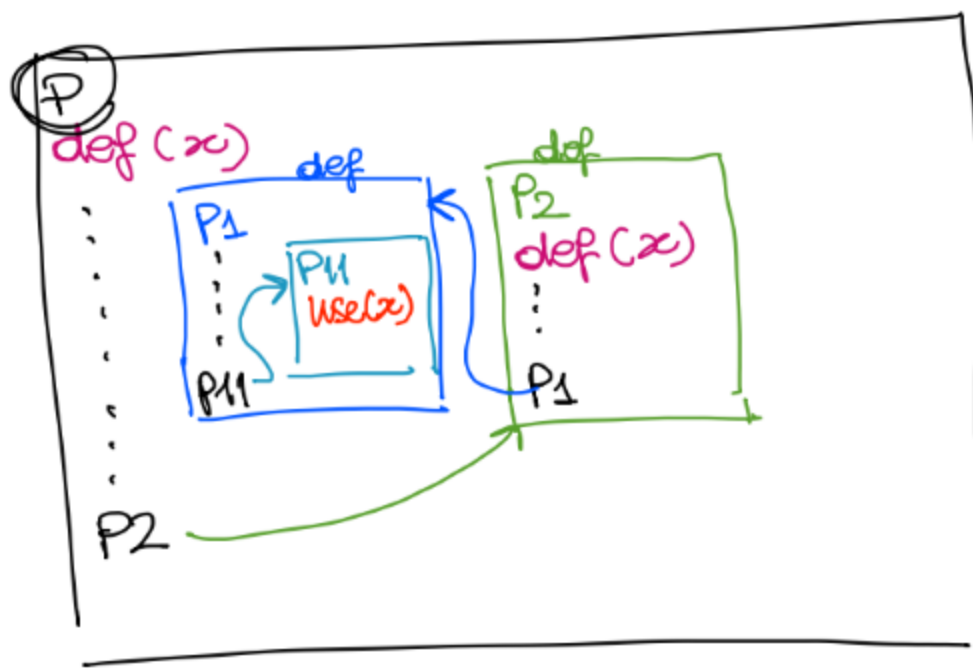
Scoping
dinamico

oppure

x = 11

Scoping
statico

SCOPING STATICO



Nella sequenza di annidamento statico (delle definizioni)

P11 NON VEDE P2

→ uso di x in P11 si riferisce alla def di x in P1



Applicheremo la regola di visibilità usando come relazione di annidamento quello di definizione

Chiamate $P \rightarrow P_2 \rightarrow P_1 \rightarrow P_M$



scoping statico

$\Rightarrow$ La risoluzione dei riferimenti avviene a tempo di compilazione

```
{ int x=0
```

```
void pippo(int m) {
```

```
    x = m+1;
```

x non locale
m locale

pippo(3); $\rightarrow$ modifica x (quella globale) $\rightarrow 4$

write(x); $\rightarrow 4$

```
{ int x=0;
```

```
    pippo(4);
```

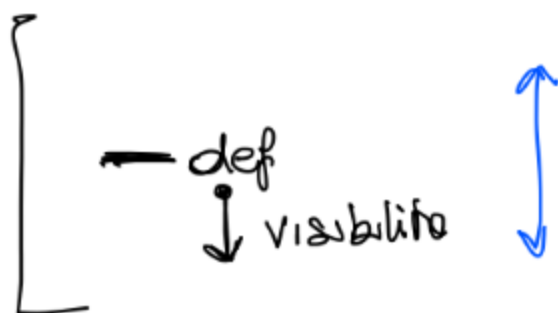
```
    write(x);
```

$\rightarrow$ modifica x (quella globale) $\rightarrow 5$

```
}
```

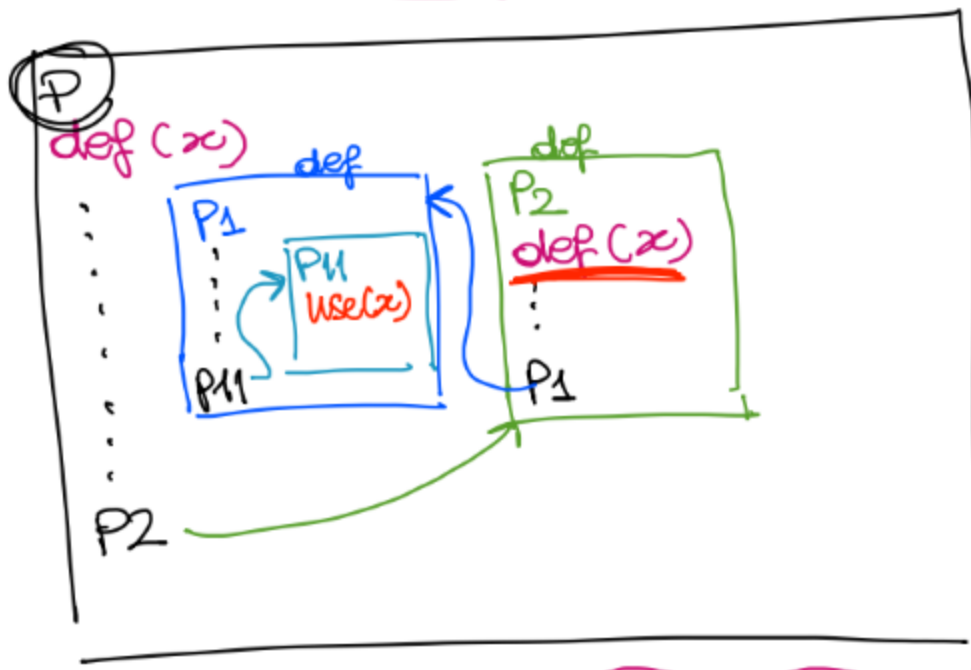
write(x); $\rightarrow 5$

$\Rightarrow$ Scoping statico garantisce l'indipendenza (del riferimento) dalla posizione della chiamata



- $\rightarrow$ rende il codice più comprensibile
- $\rightarrow$ più difficile da implementare

SCOPING DINAMICO



Chiamate : $P \rightarrow P_2 \rightarrow P_1 \rightarrow P_{11}$
 $\text{def } x$

Applica le regole di visibilità usando come "relazione di annidamento" quella della sequenza di chiamate

$\Rightarrow$ Il riferimento (lookup) corretto dipende dalla posizione della chiamata

$\Rightarrow$ Chiamate diverse fanno riferimento a definiti diversi

